openplatform v3 — 외부 서비스 API 매뉴얼

대상 서비스 (v3 Keycloak SSO 통합 5종):

1. **Rocket.Chat** — 메신저 (REST v1 + Realtime DDP)
2. **Wiki.js** — 위키 (GraphQL 전용)
3. **MinIO** — 오브젝트 스토리지 (S3 호환 + Admin API)
4. **Stalwart** — 웹메일 (Management REST + JMAP + IMAP/SMTP)
5. **LiveKit** — 화상회의 (Room Service + JWT grants)

목적: v3 포탈의 결재·게시판·캘린더 등 내부 기능 구현 시 외부 서비스를 호출할 때 참고할 실전 API 레퍼런스. 본 문서는 `docs/approval.md` 와 동일한 스타일(실측 + 표 + 예시 + 사용 가능 vs 스텁 체크리스트)을 따른다.

참고 사항

- 모든 URL 은 kc.localtest.me 단일 호스트 + v3 포트 대역(19xxx) 기준
- 모든 예시는 `admin / admin` (Keycloak), `v3admin / Admin1234!` (Rocket.Chat 로컬 관리자) 등 실제 시드 계정 사용
- BFF Port/Adapter 경유 여부 ✅ 🟡 ❌ 로 표기
- 문서 내 `<token>` 은 Keycloak access_token 또는 해당 서비스 발급 토큰

목차

1. Rocket.Chat (메신저)
2. Wiki.js (위키)
3. MinIO (스토리지)
4. Stalwart (웹메일)
5. LiveKit (화상회의)
6. BFF 어댑터 매트릭스
7. 공통 인증 플로우

---

1. Rocket.Chat (메신저)

- **서버 URL**: `http://localhost:19065`
- **버전**: 6.13.0 Community Edition
- **공식 문서**: https://developer.rocket.chat/reference/api
- **DB**: MongoDB (v3-mongo)
- **BFF Adapter**: `RocketChatAdapter.java` (대부분 stub, Phase 10 구현 예정)

1.1 인증 방식

Rocket.Chat 은 **두 가지 인증 경로**를 병행합니다.

경로	용도	헤더
Password 로그인	관리 스크립트, 초기 설정	POST `/api/v1/login` 으로 `authToken` 받아 `X-Auth-Token` + `X-User-Id` 로 전달
Custom OAuth (Keycloak)	일반 사용자 SSO	브라우저 리다이렉트 → RC 가 세션 쿠키 발급, 같은 쿠키를 REST 호출에도 사용 가능

Personal Access Token	3rd party 봇/스크립트	Admin 패널에서 발급 → `X-Auth-Token` 으로 사용

v3 권장: 사용자 컨텍스트가 필요한 호출은 Keycloak 세션 쿠키 + REST, 서버 간 배치는 `v3admin` 패스워드 토큰.

1.2 로그인 / 로그아웃

```
# Password grant
curl -X POST http://localhost:19065/api/v1/login ₩
  -H "Content-Type: application/json" ₩
  -d '{"user":"v3admin","password":"Admin1234!"}'
# → { status:"success", data:{ userId, authToken, me:{...} } }

# Logout
curl -X POST http://localhost:19065/api/v1/logout ₩
  -H "X-Auth-Token: $TOKEN" -H "X-User-Id: $UID"

# Me (현재 사용자 프로파일)
curl http://localhost:19065/api/v1/me ₩
  -H "X-Auth-Token: $TOKEN" -H "X-User-Id: $UID"
```

1.3 채널 / 그룹 / DM

메서드	엔드포인트	설명
GET	/api/v1/channels.list	공개 채널 전체
GET	/api/v1/channels.list.joined	내가 가입한 공개 채널
GET	/api/v1/channels.info?roomId=<id>	채널 상세
GET	/api/v1/channels.history?roomId=<id>&count=50&offset=0	메시지 페이징
GET	/api/v1/channels.members?roomId=<id>	참여자 목록
POST	/api/v1/channels.create {name}	공개 채널 생성
POST	/api/v1/channels.invite {roomId, userId}	사용자 초대
POST	/api/v1/channels.kick {roomId, userId}	강퇴
POST	/api/v1/channels.setTopic {roomId, topic}	주제 설정
POST	/api/v1/channels.archive {roomId}	아카이브
DELETE	/api/v1/channels.delete {roomId}	채널 삭제
GET	/api/v1/groups.list / .members / .history	**비공개** 채널 (p 접두)
POST	/api/v1/groups.create	비공개 채널 생성
POST	/api/v1/im.create {username}	1:1 DM 채널 생성
GET	/api/v1/im.list	내 DM 목록

GET	/api/v1/im.messages?roomId=<id>	DM 메시지

1.4 메시지 전송 / 수정 / 삭제

메서드	엔드포인트	설명
POST	/api/v1/chat.postMessage {channel, text, attachments[], alias}	단순 전송
POST	/api/v1/chat.sendMessage {message:{rid, msg, tmid?, attachments}}	고급 전송 (스레드 지원)
POST	/api/v1/chat.update {roomId, msgId, text}	수정
POST	/api/v1/chat.delete {roomId, msgId, asUser:true}	삭제
POST	/api/v1/chat.react {emoji, messageId, shouldReact:true}	반응 추가
POST	/api/v1/chat.pinMessage {messageId}	고정
POST	/api/v1/chat.starMessage {messageId}	즐겨찾기
POST	/api/v1/chat.reportMessage {messageId, description}	신고
GET	/api/v1/chat.getMessage?msgId=<id>	단건 조회
GET	/api/v1/chat.search?roomId=<id>&searchText=keyword	검색

스레드 (threaded replies):

```
POST /api/v1/chat.sendMessage
{
  "message": {
    "rid": "<room>",
    "tmid": "<parentMessageId>",
    "msg": "답글 본문"
  }
}
```

1.5 사용자 / 그룹 관리

메서드	엔드포인트	설명
GET	/api/v1/users.list?count=50&offset=0	사용자 목록 (admin)
GET	/api/v1/users.info?userId=<id>	사용자 상세
POST	/api/v1/users.create {email, name, password, username, roles[]}	사용자 생성
POST	/api/v1/users.update {userId, data:{...}}	수정
DELETE	/api/v1/users.delete {userId}	삭제
POST	/api/v1/users.setAvatar multipart/form-data	아바타 업로드
POST	/api/v1/users.setActiveStatus {userId, activeStatus}	활성/비활성

POST	/api/v1/users.setPassword {userId, newPassword}	비번 초기화

1.6 파일 업로드

```
curl -X POST http://localhost:19065/api/v1/rooms.upload/<roomId> ₩
  -H "X-Auth-Token: $TOKEN" -H "X-User-Id: $UID" ₩
  -F "file=@document.pdf" ₩
  -F "description=계약서 초안"
# → message 객체 + attachment URL
```

1.7 알림 / 구독

엔드포인트	설명
GET /api/v1/subscriptions.getAll	내 모든 구독 (채널별 unread 카운트) — 대시보드 배지 용
GET /api/v1/subscriptions.getOne?roomId=<id>	단건
POST /api/v1/subscriptions.read {rid}	읽음 표시
POST /api/v1/chat.syncMessages?lastUpdate=<iso>&roomId=<id>	증분 동기화

1.8 실시간 이벤트 (Realtime DDP + WebSocket)

Rocket.Chat 의 실시간 피드는 DDP (Meteor) 프로토콜을 사용합니다.

```
ws://localhost:19065/websocket
```

최소 클라이언트 플로우:

```
const ws = new WebSocket('ws://localhost:19065/websocket');
ws.onopen = () => {
  ws.send(JSON.stringify({ msg: 'connect', version: '1', support: ['1'] }));
};
ws.onmessage = (ev) => {
  const d = JSON.parse(ev.data);
  if (d.msg === 'connected') {
    // 로그인
    ws.send(JSON.stringify({
      msg: 'method', method: 'login', id: '1',
      params: [{ resume: authToken }]
    }));
    // 구독
    ws.send(JSON.stringify({
      msg: 'sub', id: '2', name: 'stream-notify-user',
      params: [`${userId}/notification`, false]
    }));
```

```
  }
};
```

주요 스트림:

- `stream-room-messages` (roomId, false) — 특정 방 새 메시지
- `stream-notify-user` (userId/notification) — 개인 알림 (멘션, DM)
- `stream-notify-user` (userId/rooms-changed) — 내 방 목록 변경
- `stream-notify-logged` (updateAvatar) — 아바타 갱신 방송

1.9 Webhook (Outgoing)

Admin > Integrations > New > Outgoing:

- Event Trigger: `Message Sent`
- Channel: `#general` 또는 `all_public_channels`
- URLs: `http://backend-bff:8080/api/bff/messenger/webhook`

Payload (JSON):

```
{
  "text": "원본 메시지",
  "user_id": "u1",
  "user_name": "홍길동",
  "channel_id": "GENERAL",
  "channel_name": "general",
  "timestamp": "2026-04-16T10:00:00.000Z",
  "token": "<outgoing token>"
}
```

1.10 v3 통합 시 유의점

- **SSO 경로**: 포탈에서 메신저 열기 → `http://kc.localtest.me:19281/realms/openplatform-v3/protocol/openid-connect/auth?client_id=rocketchat&redirect_uri=http://localhost:19065/_oauth/keycloak&...` 로 직접 진입
- **단일 MongoDB**: `mongo://mongo:27017/rocketchat` , replicaSet `rs0`
- **관리 계정**: `v3admin` / `Admin1234!` (env `ADMIN_USERNAME/ADMIN_PASS`)
- **Custom OAuth 설정 영속화**: RC 의 OVERWRITE_SETTING_Accounts_OAuth_Custom_* 환경변수는 Custom OAuth 에는 적용되지 않음 → REST API 로 설정 주입 (`POST /api/v1/settings/<key>`)

---

2. Wiki.js (위키)

- **서버 URL**: `http://localhost:19001`
- **버전**: 2.x (`ghcr.io/requarks/wiki:2`)
- **공식 문서**: https://docs.requarks.io/dev/api
- **인증 프로토콜**: Keycloak OIDC (v3) + Local (fallback)
- **API**: **GraphQL 단일 엔드포인트** `POST /graphql`
- **BFF Adapter**: `WikiJsAdapter.java` (stub)

2.1 인증 방식

Wiki.js 는 REST API 가 **없고** GraphQL + session cookie 만 지원합니다.

- 브라우저: Keycloak 로그인 → Wiki.js 가 세션 쿠키 발급 → GraphQL 호출 시 자동 전송
- 서버-서버: Admin 패널에서 **API Key** 발급 후 `Authorization: Bearer <apiKey>` 헤더로 GraphQL 호출

API Key 발급: `Administration → API Access → Enable API → New Key`

2.2 GraphQL 엔드포인트

```
curl http://localhost:19001/graphql ₩
  -H "Authorization: Bearer $WIKIJS_API_KEY" ₩
  -H "Content-Type: application/json" ₩
  -d '{"query":"{ users { profile { id email name } } }"}'
```

2.3 Pages (페이지)

```
# 페이지 목록
query {
  pages {
    list(orderBy: TITLE, limit: 50) {
      id path title description isPublished
      createdAt updatedAt authorName
    }
  }
}

# 페이지 단건 (본문 포함)
query($id: Int!) {
  pages {
    single(id: $id) {
      id path title content contentType
      description tags { tag } authorName createdAt updatedAt
    }
  }
}

# 페이지 검색
query($q: String!) {
  pages {
    search(query: $q) {
      results { id title description path locale }
      totalHits
    }
  }
}

# 페이지 트리 (사이드바)
query {
  pages {
    tree(mode: ALL, parent: 0, locale: "en", includeAncestors: false) {
      id path title isPrivate isFolder pageId parent
    }
```

```
  }
}

# 경로로 조회
query {
  pages {
    singleByPath(path: "home", locale: "en") { id title content }
  }
}
```

페이지 생성/수정:

```
mutation($content: String!, $description: String!, $editor: String!,
         $isPublished: Boolean!, $isPrivate: Boolean!, $locale: String!,
         $path: String!, $tags: [String]!, $title: String!) {
  pages {
    create(
      content: $content, description: $description, editor: $editor,
      isPublished: $isPublished, isPrivate: $isPrivate, locale: $locale,
      path: $path, tags: $tags, title: $title
    ) {
      responseResult { succeeded errorCode slug message }
      page { id path title }
    }
  }
}
```

주요 뮤테이션:

이름	파라미터	설명
pages.create	content, path, title, editor("markdown"), locale, isPublished, isPrivate, tags	신규 페이지
pages.update	id + 동일 필드	기존 페이지 수정
pages.delete	id	삭제
pages.move	id, destinationPath, destinationLocale	이동
pages.render	id	수동 재렌더링
pages.rebuildPageTree	(인수 없음)	트리 재구성

2.4 Assets (첨부 파일)

```
query {
  assets {
    list(folderId: 0, kind: ALL) {
      id filename ext kind mime fileSize metadata
      createdAt updatedAt folder { id name }
```

```
    }
  }
}

mutation($assetId: Int!) {
  assets { deleteAsset(id: $assetId) { responseResult { succeeded } } }
}
```

업로드: GraphQL multipart spec 필요 (REST 스타일 업로드는 `/u` 엔드포인트 사용 — `POST /u?folderId=0`).

```
curl -X POST http://localhost:19001/u ₩
  -H "Cookie: jwt=<session>" ₩
  -F "mediaUpload={₩"folderId₩":0}" ₩
  -F "mediaFile=@diagram.png"
```

2.5 Users / Groups

```
# 사용자 목록
query {
  users {
    list { id email name providerKey isActive lastLoginAt }
  }
}

# 사용자 상세
query($id: Int!) {
  users {
    single(id: $id) {
      id email name providerKey isActive tfalsActive
      createdAt lastLoginAt groups { id name }
    }
  }
}

# 그룹 목록
query {
  groups {
    list { id name isSystem userCount createdAt }
  }
}

# 사용자 그룹 할당
mutation($userId: Int!, $groupIds: [Int]!) {
  users {
    update(id: $userId, groups: $groupIds) {
      responseResult { succeeded }
    }
  }
}
```

2.6 Search (통합 검색)

```
query($q: String!) {
  pages {
    search(query: $q, path: "", locale: "") {
      results { id title description path locale }
      suggestions totalHits
    }
  }
}
```

Wiki.js 는 기본 DB 검색 + 엘라스틱 옵션 지원. 대규모 배포 시 Elasticsearch 모듈 활성화.

2.7 Navigation (내비게이션 트리)

```
query {
  navigation {
    tree { locale tree { id label icon targetType target kind } }
  }
}
mutation($tree: [NavigationTreeInput]!) {
  navigation { updateTree(tree: $tree) { responseResult { succeeded } } }
}
```

2.8 Authentication Strategies (v3 Keycloak 설정)

```
query {
  authentication {
    strategies {
      key strategy { key title } displayName
      isEnabled order selfRegistration domainWhitelist
      autoEnrollGroups config { key value }
    }
  }
}
```

2.9 v3 통합 시 유의점

- **API Key 발급 후** `backend-bff` `application.yml` **의** `bff.wikijs.api-token` **에 주입** → WikiJsAdapter 가 GraphQL 호출에 사용
- **autoEnrollGroups**: 현재 v3 에서는 `[2]` (Guests) 로 설정 — 신규 SSO 사용자 기본 권한
- **DB 직접 접근 주의**: `authentication.config` 컬럼의 URL 5종은 plain string 이어야 함 (`{"v":"..."}` 래퍼 금지)
- **업로드 용량**: 기본 5MB, 증가 필요 시 Admin > General > Upload Max File Size

---

3. MinIO (스토리지)

- **S3 API 포트**: `http://localhost:19900` (9000)

- **Console 포트**: `http://localhost:19901` (9001)
- **버전**: RELEASE.2023-11-20T22-40-07Z
- **호환**: AWS S3 API 100% 호환
- **BFF Adapter**: `MinioStorageAdapter.java` ✅ 사용 가능 (MinIO Java SDK)
- **공식 문서**: https://min.io/docs/minio/linux/reference/minio-api.html

3.1 인증 방식

방식	용도	사용처
Access/Secret Key	서버-서버	BFF MinioStorageAdapter (`v3minio/v3minio_pass`)
OpenID Connect	Console 사용자 로그인	Keycloak `minio` 클라이언트 (policy=consoleAdmin)
STS (Security Token Service)	임시 자격 증명	`AssumeRoleWithWebIdentity` + Keycloak JWT

3.2 Bucket 연산 (S3 REST)

```
# mc (MinIO Client) CLI 설정
mc alias set v3 http://localhost:19900 v3minio v3minio_pass

# 버킷 목록
mc ls v3/
mc ls v3/platform-v3/

# 버킷 생성
mc mb v3/approval-attachments
mc mb v3/board-files

# 버킷 정책 설정 (public read)
mc anonymous set download v3/public-files

# 버킷 삭제 (비어있어야 함)
mc rb v3/old-bucket
```

3.3 Object 연산

```
# 업로드
mc cp ./contract.pdf v3/approval-attachments/doc123/
# 업로드 (stream)
cat file.zip | mc pipe v3/approval-attachments/doc123/file.zip

# 다운로드
mc cp v3/approval-attachments/doc123/contract.pdf ./
# 파이프 다운로드
mc cat v3/approval-attachments/doc123/file.zip > local.zip

# 목록
mc ls --recursive v3/approval-attachments/doc123/
```

```
# 삭제
mc rm v3/approval-attachments/doc123/contract.pdf
mc rm --recursive --force v3/approval-attachments/doc123/

# 이동 / 복사
mc mv v3/a/file.pdf v3/b/file.pdf
mc cp v3/a/file.pdf v3/b/file.pdf
```

3.4 Presigned URL (v3 핵심 패턴)

업로드 (PUT): 브라우저가 직접 S3 에 올리고 서버는 URL 만 발급

```
// MinioStorageAdapter.java - 이미 구현됨
String url = client.getPresignedObjectUrl(
    GetPresignedObjectUrlArgs.builder()
        .method(Method.PUT)
        .bucket("platform-v3")
        .object("approval/doc123/contract.pdf")
        .expiry(10, TimeUnit.MINUTES)
        .build()
);
```

다운로드 (GET): 동일 패턴, `Method.GET`

curl 업로드 예:

```
# 1) BFF 에 presigned URL 요청
curl -H "Authorization: Bearer $JWT" ₩
     "http://localhost:19091/api/bff/storage/presigned?
object=approval/123/contract.pdf&op=PUT&expire=600"
# → { "url": "http://localhost:19900/platform-v3/approval/123/...signed...", "object": "..." }

# 2) 브라우저/클라이언트가 직접 PUT
curl -X PUT --data-binary @contract.pdf "$PRESIGNED_URL"
```

3.5 Admin API (계정/정책 관리)

`mc admin` 하위 명령 또는 REST:

```
# 사용자
mc admin user add v3 newuser newuser_pass
mc admin user list v3
mc admin user disable v3 olduser
mc admin user rm v3 olduser

# 정책 (canned policies: readonly, readwrite, writeonly, consoleAdmin)
mc admin policy create v3 approval-rw ./approval-rw-policy.json
mc admin policy attach v3 approval-rw --user newuser
mc admin policy ls v3
```

```
# 서비스 계정 (access key 페어 발급)
mc admin user svcacct add v3 v3minio --access-key batch01 --secret-key batch01secret
```

3.6 Webhook / Event Notification

버킷에 파일이 올라오면 HTTP 웹훅 호출 가능:

```
mc event add v3/approval-attachments arn:minio:sqs::primary:webhook ₩
  --event put,delete

# 사전에 server config 에서 webhook 대상 등록
mc admin config set v3 notify_webhook:primary ₩
  endpoint="http://backend-bff:8080/api/bff/storage/webhook" ₩
  queue_dir="/tmp/events"
mc admin service restart v3
```

3.7 STS — 임시 자격증명 (고급)

```
curl -X POST "http://localhost:19900" ₩
  -d "Action=AssumeRoleWithWebIdentity&DurationSeconds=3600" ₩
  -d "WebIdentityToken=$KEYCLOAK_JWT" ₩
  -d "Version=2011-06-15"
# → <AssumeRoleWithWebIdentityResult> 에 Credentials(AccessKey/SecretKey/SessionToken)
```

클라이언트가 Keycloak JWT 만 들고 있다면 MinIO 에 직접 AssumeRole 호출 가능. BFF 가 중간 매개 없이도 동작.

3.8 MinIO 버킷 명명 규칙 (v3 권장)

버킷	용도
`platform-v3`	기본 (BFF 설정)
`approval-attachments`	결재 문서 첨부
`board-files`	게시판 첨부
`calendar-attachments`	일정 첨부
`avatars`	사용자 아바타
`wiki-uploads`	Wiki.js 미러 (선택)

Object key 규칙: `<domain>/<entity_id>/<filename>` (예: `approval/123/contract.pdf`)

3.9 v3 통합 시 유의점

- **MinIO Console SSO**: 현재 `minio` Keycloak 클라이언트 + `policy=consoleAdmin` 하드코딩 매퍼 → 실제 운영에서는 사용자별 정책 매핑 필요
- **Presigned expiry 제한**: 최대 7일 (S3 호환). 실무는 10~30분 권장
- **Content-Type 헤더**: 업로드 시 반드시 `Content-Type` 지정해야 다운로드 시 브라우저가 올바른 MIME 처리
- **버킷 이름**: 소문자 + 하이픈만 (AWS S3 규칙과 동일)

4. Stalwart (웹메일)

- **SMTP**: `localhost:19025` (25)
- **IMAP**: `localhost:19143` (143)
- **Management API / JMAP**: `http://localhost:19480` (8080)
- **버전**: stalwartlabs/stalwart:latest
- **공식 문서**: https://stalw.art/docs
- **BFF Adapter**: `StalwartMailAdapter.java` (stub)
- **디렉터리 백엔드**: LDAP (openldap 컨테이너, v3 시드 사용자)

4.1 인증 방식

경로	프로토콜	자격증명
Management UI	HTTP Basic	`admin` / `admin` (fallback-admin, SHA-512)
JMAP	HTTP Basic	LDAP 사용자 (`user1` / `user1`)
IMAP/SMTP/POP3	PLAIN / LOGIN	동일 LDAP 계정
OAuth Token	JMAP Access	`/auth/token` 으로 토큰 발급 후 `Authorization: Bearer`

4.2 Management REST API (관리자)

Base: `http://localhost:19480/api` **Auth**: `Authorization: Basic <base64(admin:admin)>`

```
# 서버 상태
curl -u admin:admin http://localhost:19480/api/info
# → { version, commit, startedAt }

# 설정 조회
curl -u admin:admin http://localhost:19480/api/settings/group?prefix=server
# → [ {key:"server.listener.smtp.bind", value:"[::]:25"}, ... ]

# 설정 변경 (동적 반영)
curl -u admin:admin -X POST http://localhost:19480/api/settings ₩
  -H "Content-Type: application/json" ₩
  -d '[{"action":"set","key":"server.hostname","value":"mail.v3.local"}]'

# 큐 (발신 대기)
curl -u admin:admin http://localhost:19480/api/queue/messages
curl -u admin:admin http://localhost:19480/api/queue/messages/<id>
curl -u admin:admin -X DELETE http://localhost:19480/api/queue/messages/<id>

# 보고서 (DMARC/TLS/DKIM)
curl -u admin:admin http://localhost:19480/api/reports
curl -u admin:admin http://localhost:19480/api/reports/dmarc/<id>

# 사용자 원본 (principal)
curl -u admin:admin http://localhost:19480/api/principal?filter=type:individual
curl -u admin:admin http://localhost:19480/api/principal/admin
```

```
# 로그 검색
curl -u admin:admin "http://localhost:19480/api/logs?level=ERROR&from=2026-04-15T00:00:00Z"
```

4.3 JMAP (사용자 메일 작업, RFC 8620)

JMAP 은 RFC 표준 HTTP 기반 메일 프로토콜. 모든 작업이 단일 `POST /.well-known/jmap` 엔드포인트로 진행됩니다.

```
# 1. 세션 정보
curl -u user1:user1 http://localhost:19480/.well-known/jmap
# → { primaryAccounts, capabilities, apiUrl, downloadUrl, uploadUrl, eventSourceUrl }
```

메일함 목록:

```
POST /api/jmap
{
  "using": ["urn:ietf:params:jmap:core", "urn:ietf:params:jmap:mail"],
  "methodCalls": [[
    "Mailbox/get",
    { "accountId": "<id>", "ids": null },
    "c1"
  ]]
}
```

메일 조회 (받은편지함 최신 50):

```
[["Email/query", {
  "accountId": "<id>",
  "filter": { "inMailbox": "<inboxId>" },
  "sort": [{ "property": "receivedAt", "isAscending": false }],
  "limit": 50
}, "c1"],
 ["Email/get", {
  "accountId": "<id>",
  "#ids": { "resultOf": "c1", "name": "Email/query", "path": "/ids" },
  "properties": ["id","subject","from","receivedAt","preview","hasAttachment"]
}, "c2"]]
```

메일 발송 (Submission):

```
[["Email/set", {
  "accountId": "<id>",
  "create": {
    "draft1": {
      "mailboxIds": { "<draftsId>": true },
      "from": [{ "email": "user1@v3.local", "name": "사용자 일" }],
      "to":   [{ "email": "admin@v3.local" }],
      "subject": "테스트",
      "bodyValues": { "b1": { "value": "본문", "charset": "utf-8" } },
      "textBody": [{ "partId": "b1", "type": "text/plain" }]
```

```
      }
    }
  }, "c1"],
   ["EmailSubmission/set", {
    "accountId": "<id>",
    "create": { "s1": { "emailId": "#draft1", "envelope": {
      "mailFrom": { "email": "user1@v3.local" },
      "rcptTo":   [{ "email": "admin@v3.local" }]
    }}}
  }, "c2"]]
```

첨부 업로드:

```
POST /upload/<accountId>
Content-Type: application/pdf
Authorization: Basic ...
→ { "accountId", "blobId", "type", "size" }
```

받은 `blobId` 를 Email/set 의 `attachments:[{blobId, type, name}]` 에 지정.

4.4 IMAP / SMTP (일반 메일 클라이언트)

- **IMAP**: `imap://user1:user1@localhost:19143`
- **SMTP Submission**: `smtp://user1:user1@localhost:587`
- **STARTTLS**: 기본 지원 (dev: self-signed 인증서)

Thunderbird/Outlook 등 표준 클라이언트 그대로 사용 가능.

4.5 Managesieve (서버측 필터 규칙)

```
# 포트 4190 (v3 는 미노출)
# Sieve 스크립트 예시 - 특정 발신자를 라벨링
require ["fileinto","imap4flags"];
if header :contains "from" "boss@v3.local" {
  addflag "WWFlagged";
  fileinto "Important";
}
```

4.6 Event Source (실시간 푸시, SSE)

```
curl -u user1:user1 -N http://localhost:19480/eventsource
# → text/event-stream
#   event: state
#   data: {"Email":"abc","Mailbox":"def"}
```

클라이언트가 state 변경을 받으면 JMAP `/get` 으로 재동기화.

4.7 OAuth2 토큰 발급 (선택)

```
curl -X POST http://localhost:19480/auth/token ₩
  -H "Content-Type: application/x-www-form-urlencoded" ₩
  -d "grant_type=password&username=user1&password=user1"
# → { access_token, token_type:"Bearer", expires_in, refresh_token }
```

4.8 DMARC / DKIM / SPF 운영

```
# DKIM 키 자동 생성/로테이션
curl -u admin:admin -X POST http://localhost:19480/api/dkim/generate ₩
  -d '{"domain":"v3.local","algorithm":"ed25519-sha256"}'

# DKIM 공개키 조회 (DNS TXT 레코드용)
curl -u admin:admin http://localhost:19480/api/dkim/v3.local
# → DNS TXT 값
```

4.9 v3 통합 시 유의점

- **디렉터리 = openldap**: 사용자 추가는 OpenLDAP LDIF 주입 또는 Keycloak → LDAP 동기화. Stalwart 자체 account DB 는 사용 안함.
- **Management UI 에 SSO 는 없음**: 브라우저 리다이렉트 방식 SSO 미지원. 관리자는 `admin/admin` 폼 입력.
- **v3 PageMail**: 현재는 단순 런처 (새 창). 실제 웹메일 UI 를 포탈 내부에 만들려면 JMAP `Email/query` + `Email/get` + `Email/set` 을 BFF `StalwartMailAdapter` 에서 프록시하는 방식 권장.
- **BFF 어댑터 포트**: `application.yml` 의 `bff.stalwart.base-url` = `http://stalwart:8080` (docker network), admin-user/admin-pass 주입됨.

---

5. LiveKit (화상회의)

- **WebSocket**: `ws://localhost:19880`
- **RTC TCP 폴백**: 19881
- **RTC UDP 미디어**: 19882
- **버전**: v1.9.12 (dev 모드)
- **API Key / Secret**: `devkey` / `devsecret_v3_changeme_32chars_minimum`
- **BFF Adapter**: `LiveKitAdapter.java` ✅ 사용 가능 (JJWT HS256 서명)
- **공식 문서**: https://docs.livekit.io/reference/server/server-apis/

5.1 인증 모델

LiveKit 은 **토큰 기반**입니다 — 서버가 HS256 JWT 를 발급하면 클라이언트가 그 토큰으로 WebSocket 연결. Keycloak 과 별도, BFF 가 중개.

JWT Claims (LiveKit 규격):

```
{
  "iss": "devkey",
  "sub": "admin",            // participant identity
  "iat": 1776250000,
  "exp": 1776271600,
  "nbf": 0,
  "name": "Admin User",
```

```
  "metadata": "{₩"role₩":₩"host₩"}",
  "video": {
    "room": "v3-general",
    "roomJoin": true,
    "roomCreate": false,
    "roomList": false,
    "roomAdmin": false,
    "canPublish": true,
    "canSubscribe": true,
    "canPublishData": true,
    "canUpdateOwnMetadata": true,
    "hidden": false,
    "recorder": false
  }
}
```

BFF `/api/bff/video/token` (POST) 가 위 JWT 를 자동 생성합니다.

5.2 Room Service API (gRPC + Twirp/HTTP)

Twirp 엔드포인트는 HTTP JSON: `POST http://localhost:19880/twirp/livekit.RoomService/<Method>` **인증**: `Authorization: Bearer <admin JWT with roomCreate/roomList/roomAdmin>`

Method	입력	설명
CreateRoom	{name, emptyTimeout, maxParticipants, metadata}	사전에 방 생성 (없어도 첫 접속 시 auto-create)
ListRooms	{names:[]}	현재 활성 방 목록
DeleteRoom	{room}	방 종료 + 모든 참가자 강퇴
ListParticipants	{room}	방 참가자 목록
GetParticipant	{room, identity}	참가자 상세
RemoveParticipant	{room, identity}	강퇴
MutePublishedTrack	{room, identity, track_sid, muted}	서버측 mute
UpdateParticipant	{room, identity, metadata, permission}	권한 변경
UpdateSubscriptions	{room, identity, track_sids[], subscribe}	구독 제어
SendData	`{room, data(base64), kind(reliable	lossy), destination_sids[]}`
UpdateRoomMetadata	{room, metadata}	방 메타 업데이트 (모든 참가자 이벤트 수신)

예시 (admin JWT 발급 후):

```
ADMIN_JWT=<JWT with roomCreate=true,roomList=true>
curl -X POST http://localhost:19880/twirp/livekit.RoomService/CreateRoom ₩
```

```
  -H "Authorization: Bearer $ADMIN_JWT" ₩
  -H "Content-Type: application/json" ₩
  -d '{"name":"v3-meeting-42","emptyTimeout":300,"maxParticipants":50}'
```

5.3 Client SDK (Vue/React/JS)

```
import { Room, RoomEvent, Track } from 'livekit-client';

const room = new Room({ adaptiveStream: true, dynacast: true });

room.on(RoomEvent.ParticipantConnected, p => console.log('join', p.identity));
room.on(RoomEvent.TrackSubscribed, (track, pub, p) => {
  if (track.kind === Track.Kind.Video) track.attach(videoEl);
});
room.on(RoomEvent.DataReceived, (payload, participant, kind) => {
  const text = new TextDecoder().decode(payload);
});

// 1) BFF 로부터 토큰 + wsUrl 받기
const { token, wsUrl } = await fetch('/api/bff/video/token', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${kc.token}` },
  body: JSON.stringify({ roomName: 'v3-general', canPublish: true })
}).then(r => r.json());

// 2) 연결
await room.connect(wsUrl, token);

// 3) 로컬 트랙 발행
const tracks = await createLocalTracks({ audio: true, video: true });
for (const t of tracks) await room.localParticipant.publishTrack(t);

// 4) 화면 공유
const screenTracks = await createLocalScreenTracks();
for (const t of screenTracks) await room.localParticipant.publishTrack(t);

// 5) 데이터 채널 전송
await room.localParticipant.publishData(
  new TextEncoder().encode('hello'),
  { reliable: true }
);

// 6) 나가기
await room.disconnect();
```

5.4 Egress (녹화)

LiveKit Egress 는 **별도 서비스** (`livekit/egress:latest` 이미지). v3 에는 미설치. 설치 시:

```
# 녹화 시작
POST /twirp/livekit.Egress/StartRoomCompositeEgress
{
  "room_name": "v3-meeting",
  "layout": "grid",
  "audio_only": false,
  "file_outputs": [{
    "filepath": "recordings/v3-meeting-{time}.mp4",
    "s3": { "access_key": "...", "secret": "...", "bucket": "recordings", "endpoint":
"http://minio:9000" }
  }]
}

# 녹화 중단
POST /twirp/livekit.Egress/StopEgress { "egress_id": "..." }
```

v3 에 추가 시 MinIO 를 S3 대상으로 재사용하면 별도 스토리지 불필요.

5.5 Webhook

방 생성/참가/종료 이벤트:

```yaml
# livekit.yaml 에 추가
webhook:
  api_key: devkey
  urls:
    - http://backend-bff:8080/api/bff/video/webhook
```

수신 페이로드 (JWT 서명됨, `Authorization: <jwt>` 헤더):

```
{
  "event": "room_started" | "room_finished" | "participant_joined" | "participant_left"
        | "track_published" | "track_unpublished" | "egress_started" | "egress_ended",
  "room": { "sid", "name", "empty_timeout", "creation_time", "num_participants" },
  "participant": { "sid", "identity", "state", "permission", "joined_at" },
  "id": "evt_...",
  "created_at": 1776250000
}
```

5.6 Ingress (외부 스트림 → 방으로)

Ingress 서비스(별도 컨테이너)로 RTMP/WHIP 입력을 LiveKit 방에 참가자로 주입 가능. 스트리밍 송출 시나리오용. v3 미설치.

5.7 v3 통합 시 유의점

- **dev 모드 제약**: in-memory 상태. Redis 모드로 바꾸면 멀티 노드 + 영속성. `livekit.yaml` 에 `redis.address: redis:6379` .
- **TURN 서버**: dev 모드는 STUN 만 제공. NAT 환경에서 원격 참가자 연결 시 TURN 필수 (`livekit-server --turn`).
- **token expiry**: BFF 기본 6시간. 장시간 회의는 `exp` 연장 필요.

- **room name = 1:1**: 동일 이름으로 참가하면 같은 방. 사전 매칭(회의록/초대 테이블) 으로 중복 방 방지.
- **SSO 불필요**: Keycloak 클라이언트는 `bearerOnly: true` (명목). 실제 토큰 발급은 BFF 가 담당.

---

6. BFF 어댑터 매트릭스

`backend-bff` 의 Port/Adapter 및 각 서비스 호출 상태.

Port	Adapter 파일	상태	구현 메서드	다음 작업
`IdentityPort`	`KeycloakIdentityAdapter`	✅ 구현	`getMe, getRoles, getUserById`	그룹/역할 조회 추가
`MessagingPort`	`RocketChatAdapter`	❌ Stub	(전부 stub 반환)	§1.3~1.7 매핑
`MailPort`	`StalwartMailAdapter`	❌ Stub	(전부 stub)	§4.3 JMAP 호출
`WikiPort`	`WikiJsAdapter`	❌ Stub	(전부 stub)	§2.3 GraphQL 호출
`StoragePort`	`MinioStorageAdapter`	✅ 구현	`uploadFile, presignedGetUrl, presignedPutUrl, removeObject`	버킷별 정책 분기
`VideoPort`	`LiveKitAdapter`	✅ 구현	`createRoom, issueToken`	Twirp RoomService 연동
`NotificationPort`	(미구현)	❌ 미구현	—	backend-core 알림 API 프록시

BFF REST 엔드포인트 (`BffController.java`):

```
GET  /api/bff/identity/me
GET  /api/bff/messenger/channels
GET  /api/bff/messenger/messages?channelId=&limit=
POST /api/bff/messenger/messages            { channelId, text }
GET  /api/bff/mail/mailbox?folder=&limit=
POST /api/bff/mail/send                     { to, subject, body, ... }
GET  /api/bff/wiki/search?keyword=
GET  /api/bff/wiki/page?pageId=
POST /api/bff/video/room                    { roomName }
POST /api/bff/video/token                   { roomName, canPublish }
GET  /api/bff/video/config
GET  /api/bff/storage/presigned?object=&op=&expire=
```

---

7. 공통 인증 플로우

7.1 Keycloak 중심 SSO (브라우저 → 서비스 직접)

```
Browser ──1──▶ Portal UI (localhost:19173)
          2   /auth?client_id=v3-ui → kc.localtest.me:19281
          3   login + consent → code → portal
          4   axios → backend-core with Bearer JWT

Browser ──5──▶ "메신저" 버튼 클릭
          6   /auth?client_id=rocketchat → kc (same cookie! auto-consent)
          7   redirect → RC /_oauth/keycloak?code=...
          8   RC server-side token exchange (via kc.localtest.me)
          9   RC 세션 쿠키 발급 → /home
```

7.2 BFF 중심 (포탈 내부 UI → 서비스 프록시)

```
Portal UI ──1──▶ /api/bff/mail/mailbox (Bearer portal JWT)
             2  BFF verifies JWT (Keycloak JWK)
             3  BFF → StalwartMailAdapter
             4  JMAP call with admin-basic or user token
             5  JSON response → UI
```

이 패턴은 포탈 내부에 **자체 UI** 를 만들 때 사용. 외부 탭을 열지 않음.

7.3 v3 서비스별 권장 패턴

서비스	권장 플로우	이유
Rocket.Chat	브라우저 직접 (Custom OAuth)	RC 자체 UI 가 풍부, 재구현 불필요
Wiki.js	브라우저 직접 (OIDC)	동일 이유
MinIO	BFF presigned	일반 사용자는 MinIO Console 안 씀, 포탈이 업/다운로드 대행
Stalwart	양방향 — 관리자는 Management UI 직접, 일반 사용자는 BFF 프록시 UI	웹메일 UI 가 없음, 포탈이 JMAP 프록시
LiveKit	BFF 토큰 발급 후 브라우저가 직접 WS	Keycloak 세션 불필요

---

8. 참고 링크

- Rocket.Chat API: https://developer.rocket.chat/reference/api
- Wiki.js API: https://docs.requarks.io/dev/api
- MinIO: https://min.io/docs/minio/linux/reference/minio-api.html
- MinIO mc CLI: https://min.io/docs/minio/linux/reference/minio-mc.html
- Stalwart docs: https://stalw.art/docs
- JMAP RFC 8620: https://jmap.io/spec-core.html
- LiveKit Server APIs: https://docs.livekit.io/reference/server/server-apis/
- LiveKit Client SDK: https://docs.livekit.io/client-sdk-js/

---

9. 개발 로드맵 요약 (v3 확장 시)

우선순위	작업	영향 파일
🔥 1	결재 첨부 구현 — MinIO presigned + `ap_attachment` 테이블	`MinioStorageAdapter`, `BffController.presigned`, `V8__approval_attachments.sql`
🔥 2	포탈 내부 웹메일 UI (JMAP 프록시)	`StalwartMailAdapter` 구현, `PageMail.vue` CRUD
🔥 3	Rocket.Chat 뱃지 (대시보드 메신저 unread)	`RocketChatAdapter.unreadBadge`, `PageDashboard.vue`
⭐ 4	Wiki.js 검색 + 본문 임베드	`WikiJsAdapter.searchPages/getPage`, `PageWiki.vue` 검색 패널
⭐ 5	LiveKit 방 사전 예약 + 초대 알림	`VideoPort.createRoom` Twirp 연동, `NotificationService` 연계
🧊 6	LiveKit Egress (회의 녹화 → MinIO)	`livekit-egress` 컨테이너 추가, BFF 녹화 시작/중단 API
🧊 7	Stalwart DKIM/DMARC 자동 설정	`StalwartMailAdapter.admin` + cron 회전

문서 끝 — 각 섹션의 curl/GraphQL/Twirp 예시는 실제 v3 환경에서 바로 실행 가능.